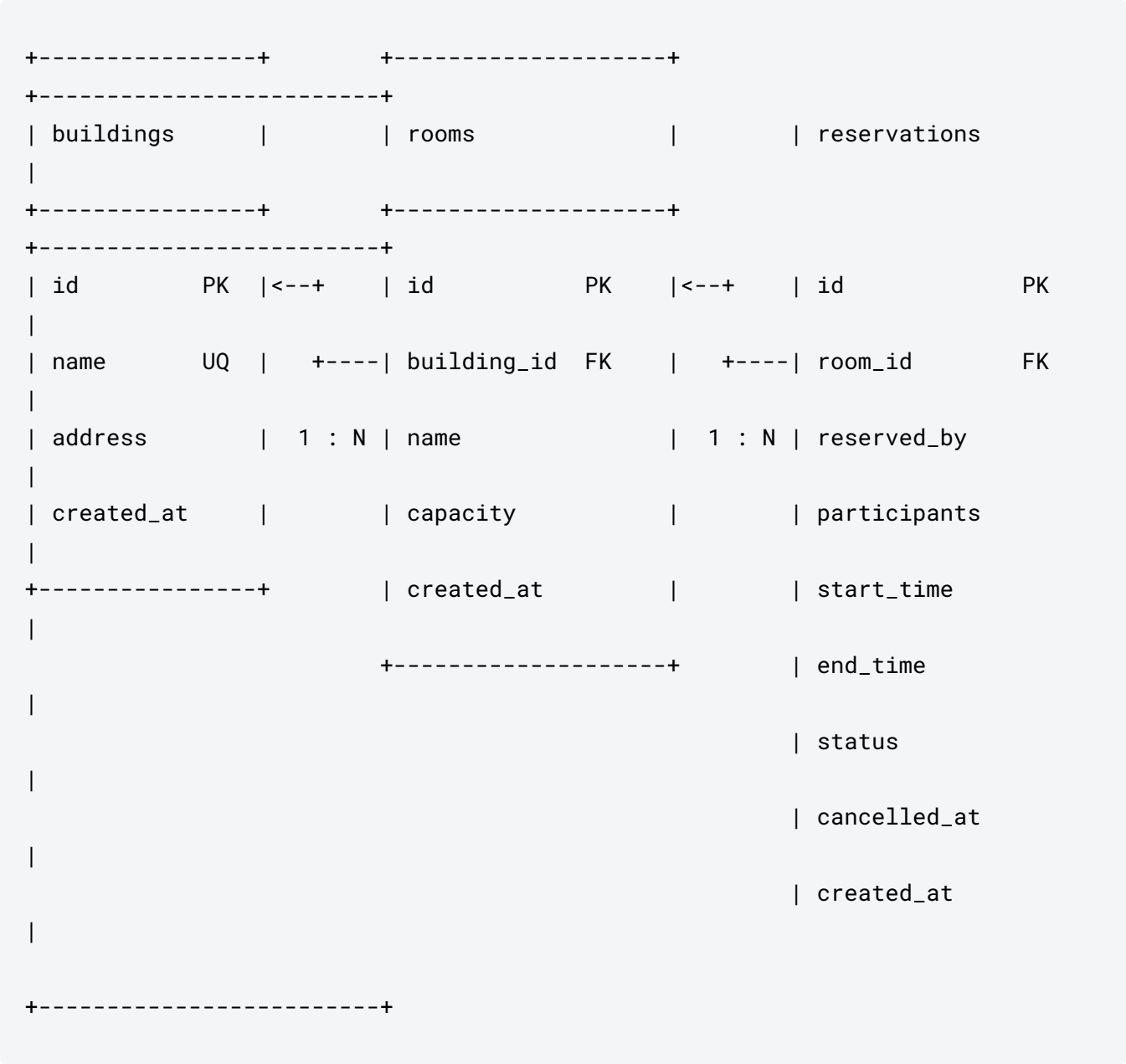
Zadanie 03 - Room Booking API: PostgreSQL, relacje i współbieżność

Założenia

Rozwijamy Room Booking API z zadania 02. Tablice w pamięci zastępujemy bazą **PostgreSQL**.

Dochodzi nowy zasób - **budynek**. Każda sala należy do jednego budynku.



Przykładowy budynek:

```
{
  "id": 1,
  "name": "Budynek A",
  "address": "ul. Warszawska 24, Kraków"
}
```

Wymagane technologie: Node.js, ExpressJS 5, Zod, PostgreSQL (w Dockerze), Knex.

> Jeżeli chcesz użyć innego narzędzia do dostępu do bazy (np. Prisma, TypeORM, Sequelize), napisz wcześniej maila. Wymagania merytoryczne (migracje, transakcje, blokady) pozostają takie same.

Za cały zestaw można uzyskać maksymalnie **5 punktów**.

Każde zadanie jest warte **1 punkt**.

Zadanie 1 - Baza w Dockerze i migracje - 1 pkt

Wymagania

- PostgreSQL uruchomiony w kontenerze Docker, z konkretną wersją obrazu (np. `postgres:16`) i nazwanym volume,
- dane do połączenia w zmiennej środowiskowej `DATABASE_URL`, plik `.env` dodany do `.gitignore`,
- plik `.env.example` z przykładowymi (nieprawdziwymi) wartościami, który **jest** w repozytorium,
- schemat bazy tworzony **wyłącznie** przez migracje Knex (żadnych `CREATE TABLE` wpisywanych ręcznie),
- tabele `buildings`, `rooms`, `reservations` zgodne z diagramem,
- constraints w bazie:
  - `buildings.name` - `UNIQUE`,
  - `rooms` - para `(building_id, name)` unikalna (w dwóch budynkach może być sala "101", w jednym - nie),

    - `rooms.capacity` - `CHECK (capacity > 0)`,
    - `reservations.end_time > reservations.start_time` - `CHECK`,
    - `reservations.status` - `CHECK` z dozwolonymi wartościami,
    - wszystkie FK z przemyślaną akcją `ON DELETE` (uzasadnij wybór w sprawozdaniu: co ma się stać z rezerwacjami po usunięciu sali?),
- kolumny z datami jako `timestamptz`,
- indeks pod zapytanie sprawdzające kolizje rezerwacji (`room_id`, `start_time`, `end_time` lub inny uzasadniony),
- `migrate:rollback` a następnie `migrate:latest` działa bez błędów (funkcje `down` są poprawne).

W sprawozdaniu pokaż kod migracji, wynik `\d reservations` z `psql` oraz próbę złamania **każdego** constraintu ręcznym `INSERT` w `psql`.

Zadanie 2 - Seeds i repositories na Knex - 1 pkt

Wymagania

- seed tworzy minimum 2 budynki, 5 sal i 5 rezerwacji (w tym jedną `CANCELLED` i jedną, która już się rozpoczęła),
- `seed:run` uruchomiony wielokrotnie daje zawsze ten sam stan (ID zaczynają się od 1),
- nowe repositories (`*.repository.knex.js`) implementują **ten sam interfejs** co repositories in-memory z zadania 02,
- repositories są **jedynym** miejscem, które zna nazwy tabel i kolumn; mapowanie `snake_case` ↔ `camelCase` odbywa się w repository,
- **logika biznesowa w services nie zmienia się** (dopuszczalna jest zmiana sposobu przekazywania repositories na `runInTransaction`),
- wybór implementacji (Knex / in-memory) odbywa się **tylko** w `server.js`,
- repositories in-memory **zostają** w projekcie (przydadzą się w zestawie 05),
- dane przetrwają restart servera,
- nowe endpointy CRUD dla budynków: `GET /api/buildings`, `GET /api/buildings/:id`, `POST /api/buildings` (z walidacją Zod),
- `POST /api/rooms` przyjmuje `buildingId`; nieistniejący budynek → `404`, duplikat nazwy sali w budynku → `409` (obsłuż błąd PostgreSQL `23505` w error handlerze, bez wycieku szczegółów

bazy do klienta).

W sprawozdaniu pokaż `git diff` (albo porównanie) plików service przed i po zmianie oraz screenshot potwierdzający, że dane przetrwały restart servera.

Zadanie 3 - Relacje i JOIN bez N+1 - 1 pkt

Wymagania

- `GET /api/rooms/:id` zwraca salę razem z budynkiem:

```
{
  "id": 3,
  "name": "101",
  "capacity": 30,
  "building": { "id": 1, "name": "Budynek A" }
}
```

- `GET /api/buildings/:id/rooms` zwraca sale danego budynku (nieistniejący budynek → `404`, budynek bez sal → `[]`),
- `GET /api/reservations/:id` zwraca rezerwację z osadzoną salą i budynkiem:

```
{
  "id": 7,
  "room": { "id": 3, "name": "101", "building": { "id": 1, "name": "Budynek
A" } },
  "reservedBy": "Jan Kowalski",
  "participants": 20,
  "startTime": "2026-11-10T10:00:00.000Z",
  "endTime": "2026-11-10T12:00:00.000Z",
  "status": "CONFIRMED",
  "createdAt": "2026-10-27T08:00:00.000Z",
  "cancelledAt": null
}
```

- `GET /api/reservations` zwraca listę w tym samym formacie,
- **każdy** z powyższych endpointów wykonuje **stałą liczbę zapytań** SQL, niezależną od liczby zwracanych elementów (brak N+1),
- `GET /api/buildings` zwraca dla każdego budynku pole `roomsCount` (liczba sal) - również bez N+1; budynek bez sal ma `roomsCount: 0`.

> **Podpowiedź:** `roomsCount` z budynkami bez sal wymaga `LEFT JOIN` + `GROUP BY` (albo podzapytania). `INNER JOIN` "zgubi" budynki bez sal.

W sprawozdaniu pokaż logi zapytań (`DEBUG=knex:query`) dla `GET /api/reservations` przy 5 i przy 20 rezerwacjach - liczba zapytań musi być taka sama.

Zadanie 4 - Pagination, filtrowanie i sortowanie - 1 pkt

Rozbuduj `GET /api/reservations`.

```
GET /api/reservations?
roomId=3&status=CONFIRMED&from=2026-11-01T00:00:00Z&to=2026-11-30T23:59:59Z&pa
ge=2&limit=10&sort=-startTime
```

Wymagania

- parametry (wszystkie opcjonalne):

Parametr	Znaczenie	Domyślnie
`roomId`	tylko rezerwacje danej sali	-
`buildingId`	tylko rezerwacje sal w danym budynku	-

`status`	`CONFIRMED` / `CANCELLED`	-
`from`, `to`	rezerwacje, które **nakładają się** na przedział `[from, to)`	-
`page`	numer strony (min 1)	`1`
`limit`	rozmiar strony (1-100)	`20`
`sort`	`startTime`, `-startTime`, `createdAt`, `-createdAt`, `participants`, `-participants`	`startTime`

- `from` / `to` działają według **tej samej** reguły nakładania się przedziałów co w zadaniu 02 (rezerwacja `10:00-12:00` jest widoczna dla `from=11:00&to=11:30`), a `from >= to` → `400`,
- filtrowanie, pagination i sortowanie wykonuje **baza danych** (`WHERE`, `ORDER BY`, `LIMIT`, `OFFSET`), a nie JavaScript na pobranej tablicy,
- sortowanie tylko po kolumnach z whitelisty; nieznana wartość `sort` → `400`,
- stabilna kolejność (tie-breaker po `id`),
- odpowiedź w formacie:

```
{
  "data": [ ... ],
  "meta": { "page": 2, "limit": 10, "total": 37, "totalPages": 4 }
}
```

- `total` uwzględnia filtry (to liczba pasujących rekordów, a nie wszystkich w tabeli),
- strona poza zakresem (np. `page=99`) → `200` i pusta tablica `data`.

W sprawozdaniu pokaż co najmniej 4 różne kombinacje parametrów oraz wygenerowane zapytania SQL.

Zadanie 5 - Double booking: race condition - 1 pkt

Reguła z zadania 02 ("sala nie może mieć dwóch nakładających się rezerwacji `CONFIRMED`") jest w tej chwili sprawdzana wzorcem **check-then-act**. Przy równoległych requestach to nie wystarcza.

Część A - pokaż problem

Napisz skrypt `scripts/double-booking.js`, który wysyła **równolegle 10 identycznych** requestów `POST /api/reservations` (ta sama sala, ten sam przedział czasu) i wypisuje:

- status code każdego requestu,
- liczbę rezerwacji `CONFIRMED` dla tej sali i tego przedziału w bazie (zapytaniem SQL po zakończeniu).

Uruchom skrypt na **obecnej** implementacji. Jeżeli problem nie występuje samoczynnie, dodaj **tymczasowe** opóźnienie między sprawdzeniem kolizji a zapisem (jak w materiale 03) i pokaż, że w bazie powstało więcej niż jedna rezerwacja.

Część B - napraw

Zastosuj **oba** zabezpieczenia:

1. **W aplikacji:** sprawdzenie kolizji i zapis wykonywane w **jednej transakcji**, z blokadą `SELECT ... FOR UPDATE` na odpowiednim wierszu (uzasadnij, który wiersz blokujesz i dlaczego),
2. **W bazie:** exclusion constraint (`btree_gist` + `tstzrange(...) WITH &&`) uwzględniający **tylko** rezerwacje `CONFIRMED`, dodany **nową migracją**; naruszenie constraintu (kod `23P01`) mapowane na `409 ROOM_ALREADY_BOOKED`.

Wymagania

- po naprawie skrypt (z tym samym sztucznym opóźnieniem) daje dokładnie **jeden** `201` i dziewięć `409`,
- po wyłączeniu sprawdzenia w aplikacji (tymczasowo) constraint w bazie nadal blokuje duplikaty - nadal jeden `201`, dziewięć `409`, **żadnego** `500`,
- anulowana rezerwacja nie blokuje utworzenia nowej w tym samym przedziale,
- rezerwacje stykające się (`10:00-12:00` i `12:00-13:00`) są dozwolone również na poziomie constraintu.

W sprawozdaniu:

- wyniki skryptu w trzech wariantach: przed naprawą, po naprawie, z wyłączonym sprawdzeniem w aplikacji,
- krótkie wyjaśnienie (kilka zdań), **dlaczego** sama transakcja bez blokady nie rozwiązuje problemu przy domyślnym poziomie izolacji PostgreSQL.

Na koniec usuń sztuczne opóźnienie z kodu.

Punktacja

Zadanie	Punkty
Baza w Dockerze i migracje	1 pkt
Seeds i repositories na Knex	1 pkt
Relacje i JOIN bez N+1	1 pkt
Pagination, filtrowanie, sortowanie	1 pkt
Double booking: race condition	1 pkt
Razem	**5 pkt**

Co oddać?

Przygotuj sprawozdanie w formacie PDF.

Powinno ono zawierać:

- fragmenty najważniejszego kodu (migracje, repository z zapytaniami, transakcja z blokadą),

- screenshoty potwierdzające działanie endpointów,
- logi wygenerowanych zapytań SQL tam, gdzie wymaga tego zadanie,
- wyniki skryptu `double-booking.js`,
- krótki opis rozwiązania i odpowiedzi na pytania z treści zadań.

Nie ma potrzeby umieszczania screenshotu całego kodu.

Najważniejsze jest pokazanie, że API działa zgodnie z wymaganiami.

Link do repozytorium GitHub - opcjonalnie (zalecany).